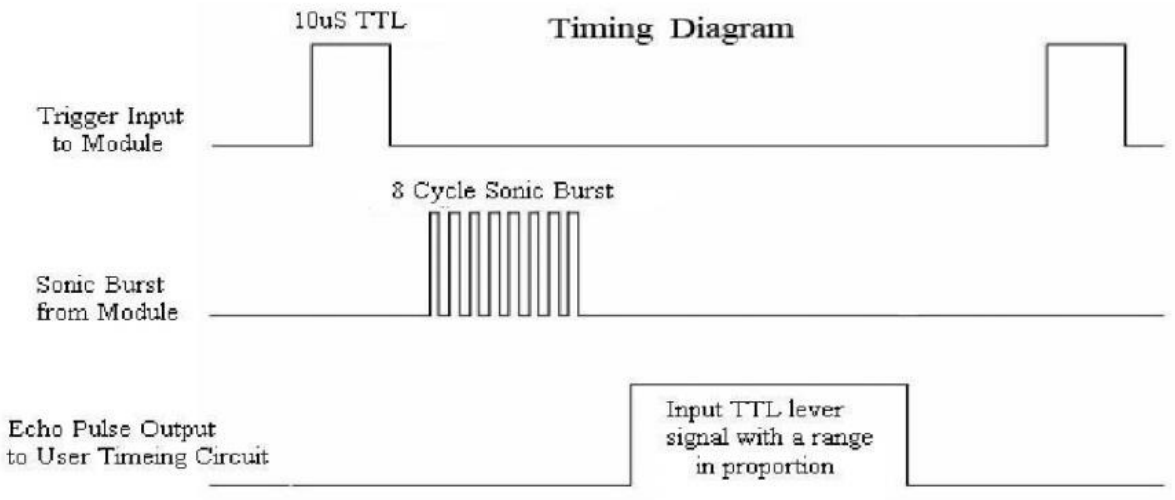
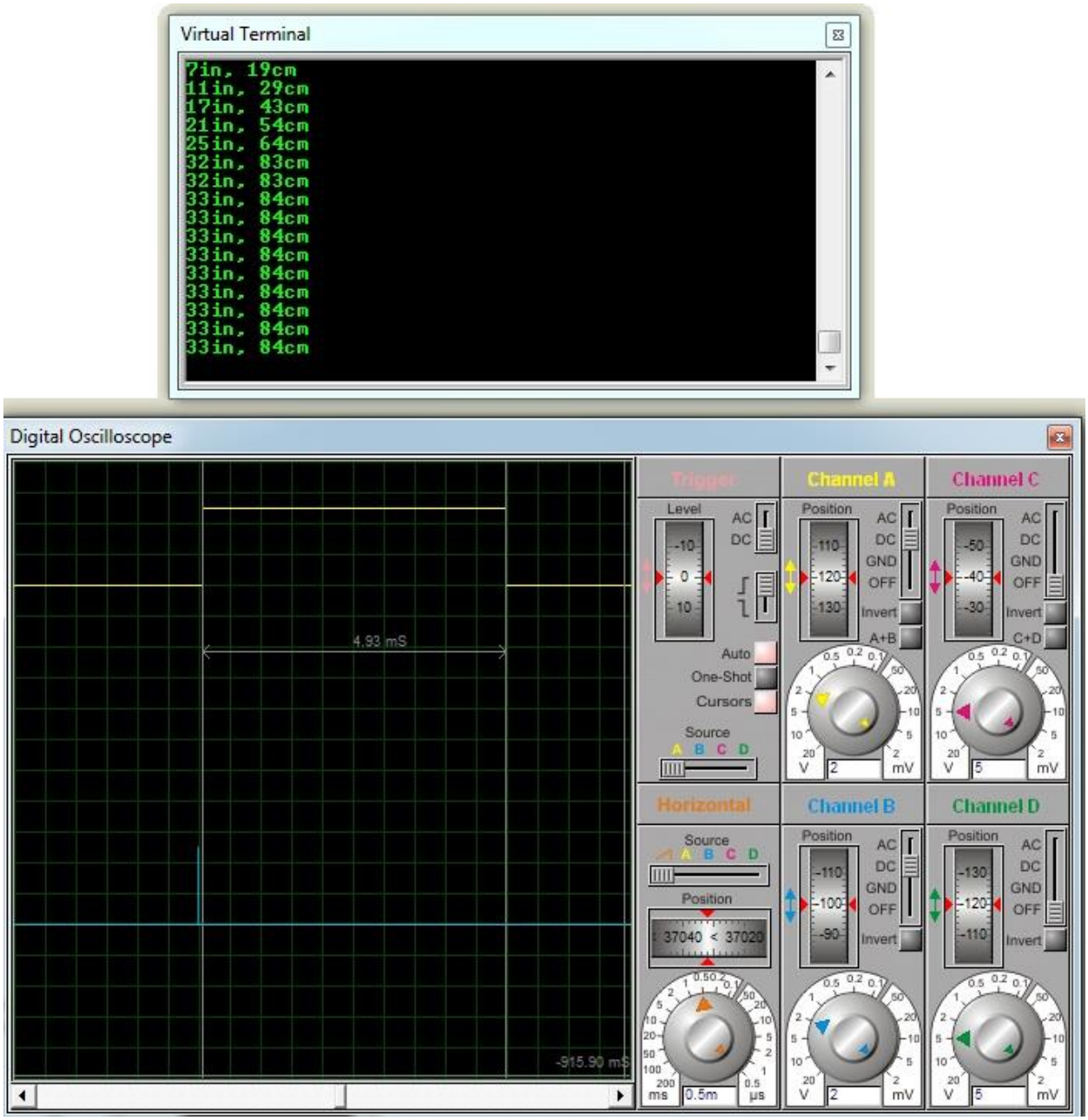
Para quem não conhece ainda este sensor, é bem fácil de entender o seu funcionamento.

Conforme diagrama mais adiante, o que acontece é que este sensor possui um microcontrolador responsável por gerar os pulsos ultrassônicos que serão emitidos através de um destes transdutores instalados no shield do sensor. E após a emissão deste pulso, ele fica aguardando o retorno do mesmo, para que seja interpretada a distância que este pulso percorreu até o objeto.
Dependendo do tempo entre o pulso emitido e o pulso recebido pelo outro transdutor, o microcontrolador do sensor vai gerar outro pulso chamado de ECHO para a placa de controle.



O segredo está na duração do pulso que o sensor gera para a placa de controle.
Como alguns sensores tem características muito semelhantes, resolvi criar um programa para um microcontrolador (PIC nesta primeira versão) ,a simulação deste pulso de ECHO.



Explicando um pouco sobre o funcionamento, o sensor Ultrassônico fica aguardando um pulso de Trigger (gatilho) para simular a recepção do pulso e então envia um pulso de ECHO para a placa de controle por um período de tempo. A placa de controle é responsável por interpretar a distância. Então para isto ela efetua a medição da duração do pulso de ECHO, que entra na seguinte fórmula:

Distância em cm = (Duração do Pulso em microssegundos) / 58

ou

Distância em polegadas = (Duração do Pulso em microssegundos) / 148

Pegando o exemplo da figura acima, vemos que o pulso teve duração de **4,93ms**, ou seja, esse pulso corresponde a **4930us.**

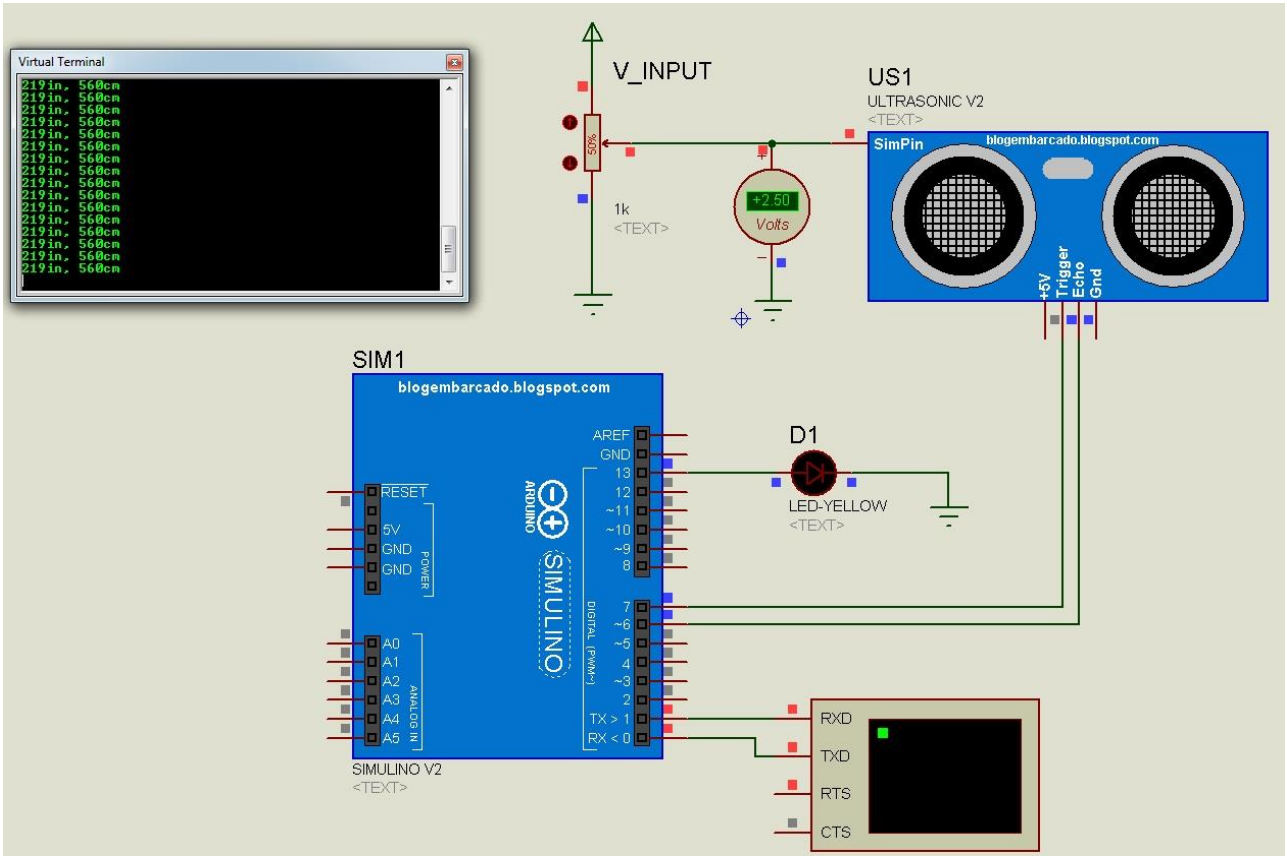
Para termos o valor em **centímetros**, devemos dividir este valor por **58**, resultando em: **85 cm**

Então alguém me pergunta: "Mas no diagrama não está **84cm**?" Responderei: "Sim meu caro Sherlock Holmes!rs", o que acontece é que esta pequena diferença é devido aos erros de aproximação nos cálculos e também pelo fato de que para a medição do pulso de entrada no microcontrolador na placa de controle, existe uma precisão agregada à configuração dos timers internos e também ao sistema de clock do mesmo.
Examinando o programa de exemplo, vemos a seguinte chamada de função:

```
duration = pulseIn(echoPin, HIGH);
```

Sendo que para saber qual será o erro agregado ao valor dado pela função "pulseIn", devemos abrir a biblioteca do mesmo e estudar a configuração de timer e clock.

Vamos simular?

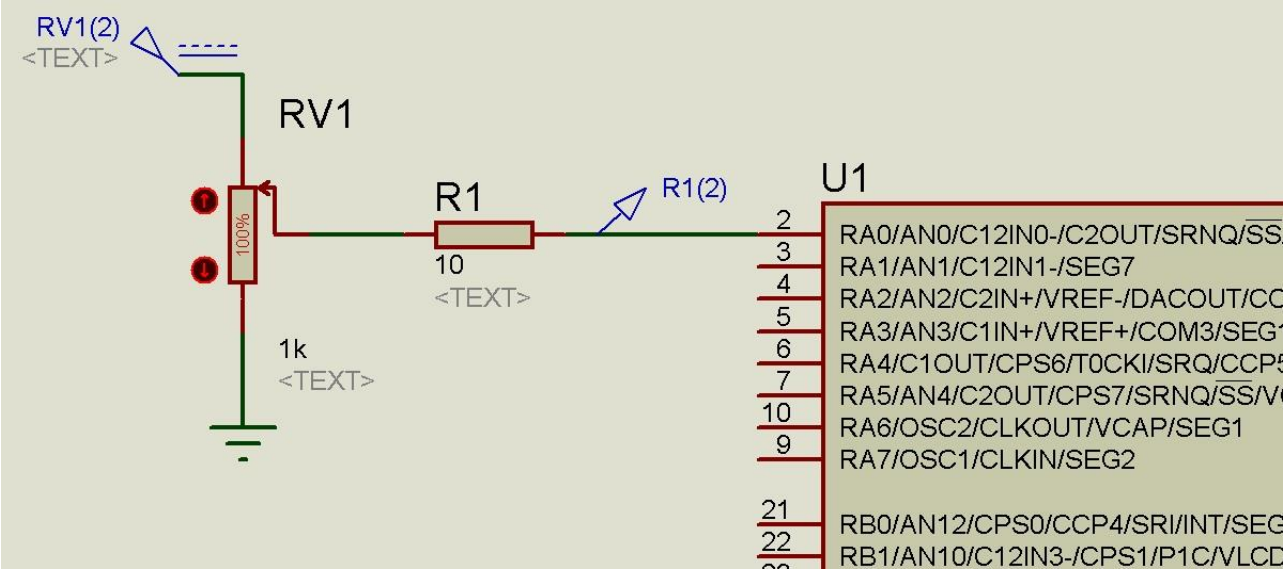


Funcionamento do Sensor Simulado:

Para facilitar o uso deste simulador no Proteus, e para que possamos ajustar a distância do

nosso objeto simulado, fiz nesta primeira versão a referência como sendo um **valor de tensão** ou sinal analógico diretamente acoplado em um dos pinos do microcontrolador PIC16.

Basicamente o que ele faz é transformar a **referência em volts** para um **valor de distância**, gerando um **pulso de ECHO**proporcional na saída. Sendo que para ele emitir esse pulso de ECHO é necessário manter o pino de **Trigger em nível alto** por pelo menos **10us** (microssegundos), conforme sensores comuns.



Para sabermos o valor de tensão que deve ser aplicado, existe uma pequena fórmula:

Valor de tensão em Volts = (Valor da distância em cm) / 225,76

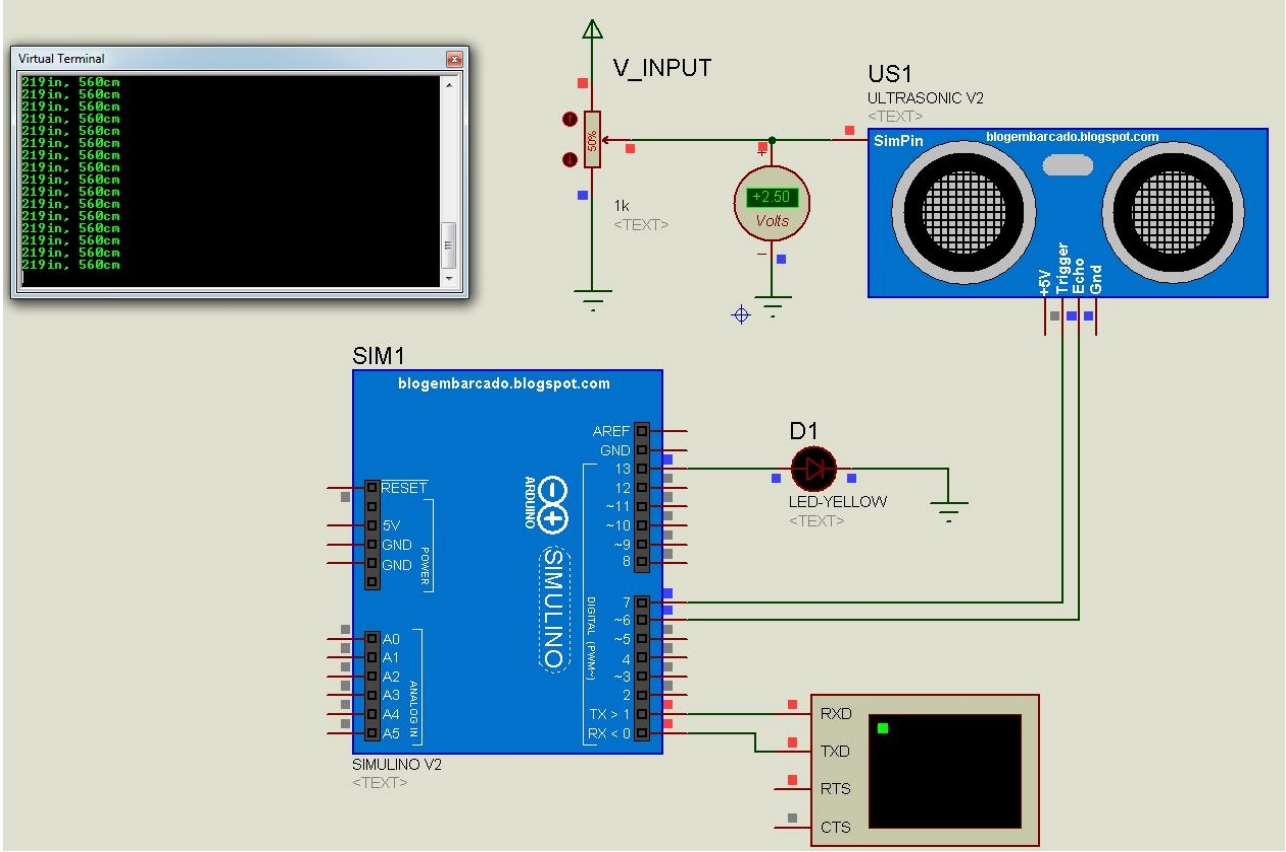
Por exemplo, queremos simular uma distância de **5m** (metros), que corresponde a **500cm** (centímetros).
Para ter-se o valor de tensão, faremos: **500 / 225,76 = 2,21 V (volts)**

Ou seja, tem-se que colocar o valor de 2,21V na entrada do pino RA0 (pino 2) do microcontrolador do sensor.
O interessante é que como é um valor analógico, fica fácil de criar diversas configurações de entrada, para simular por exemplo um objeto se afastando ou se aproximando da sua placa de controle ou robô. Que é o mais desejado.

Então, após configurado o valor de tensão para a distância do objeto, basta carregar no microcontrolador PIC16F1936 do sensor o arquivo **UltraSonicSensorPIC16F.HEX** que está dentro da pasta Source do projeto.
E na placa do simulino, pode ser utilizado o programa de exemplo incluído no projeto, devendo o mesmo ser compilado, ou então utilizado o arquivo **UltraSonic_Shield.cpp.hex.** Devendo o mesmo ser carregado no microcontrolador ATmega do Simulino.

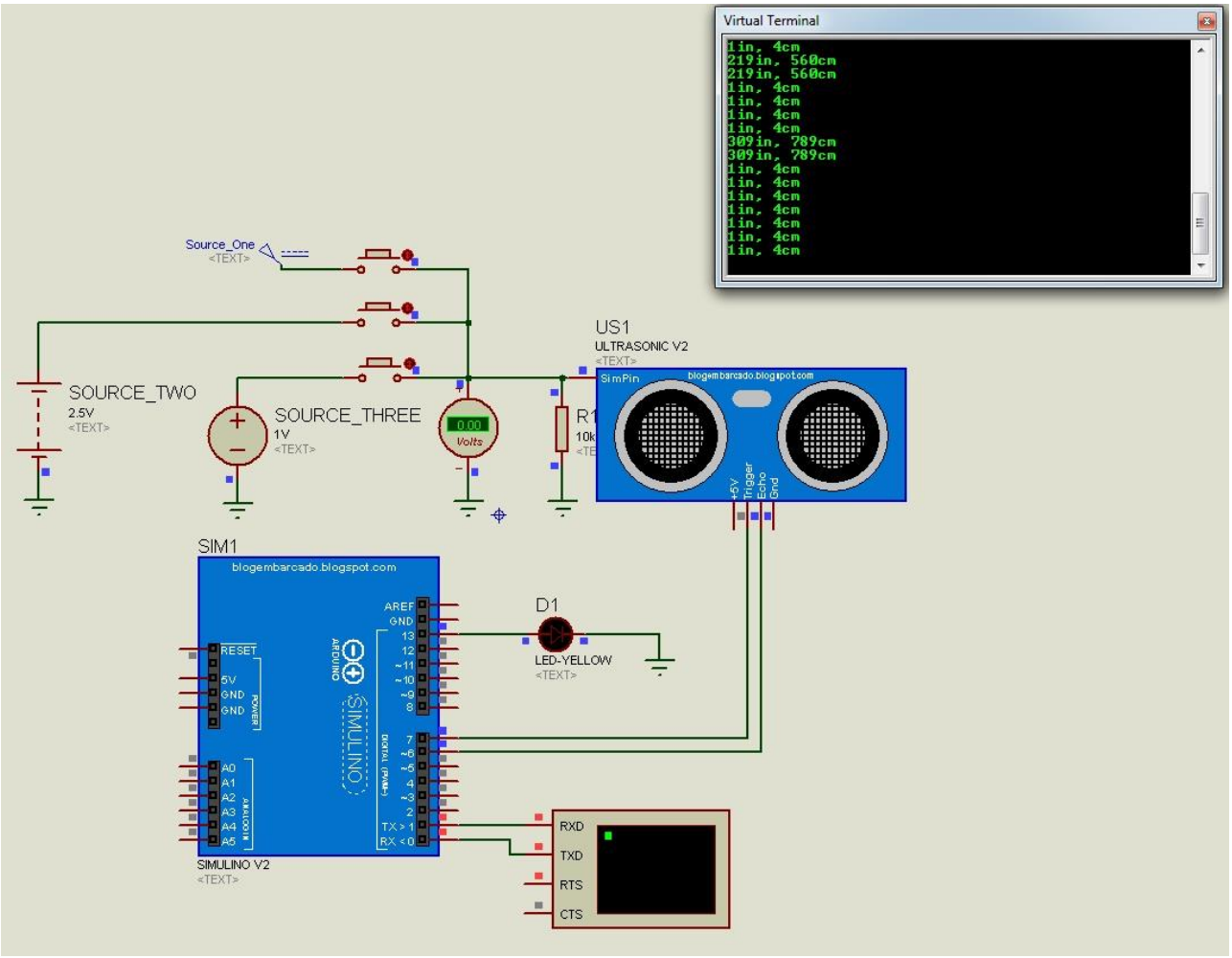
Exemplos de Como simular a aproximidade:

1) UltraSonic Simple



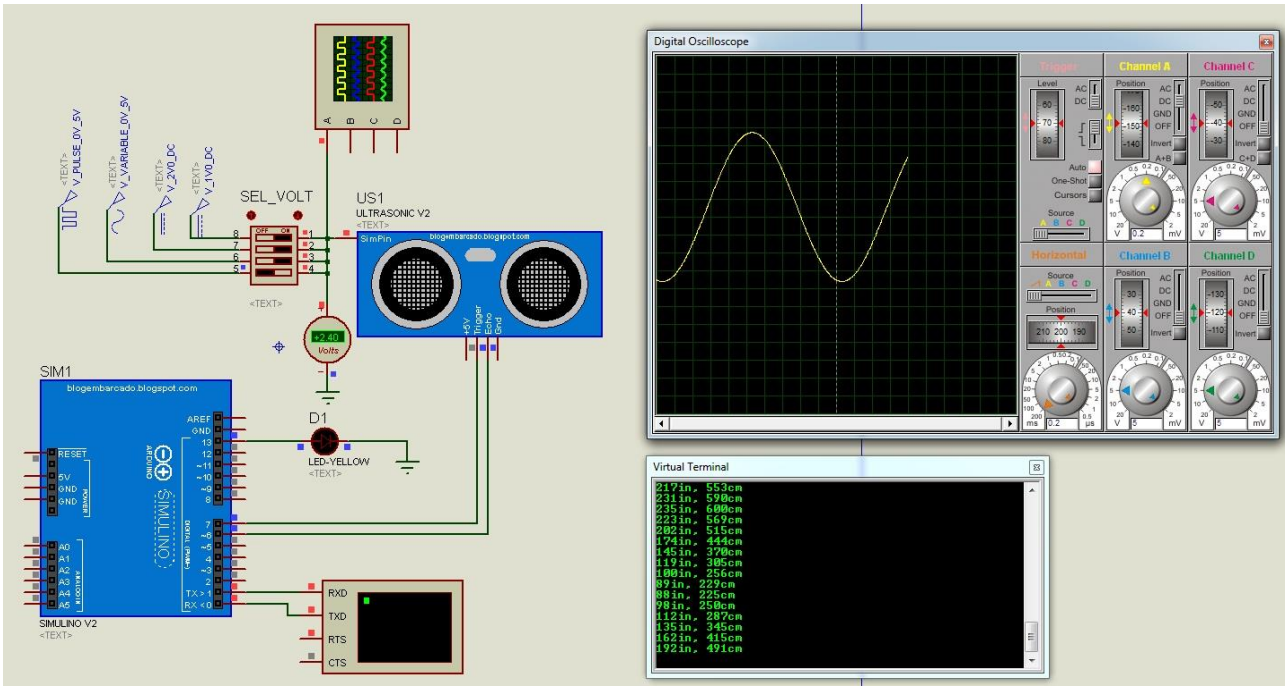
Neste exemplo é utilizado um potenciômetro, para que a medida em que ele for tendo sua porcentagem alterada, ele varie a tensão de entrada no pino do Sensor Ultrassônico, e como explicado no outro post, o sensor irá atualizar o novo valor da distância identificada pelo mesmo. E essa nova distância aparece automaticamente no Terminal Virtual, que nada mais é do que nossa serial de debug.

2) UltraSonic Button



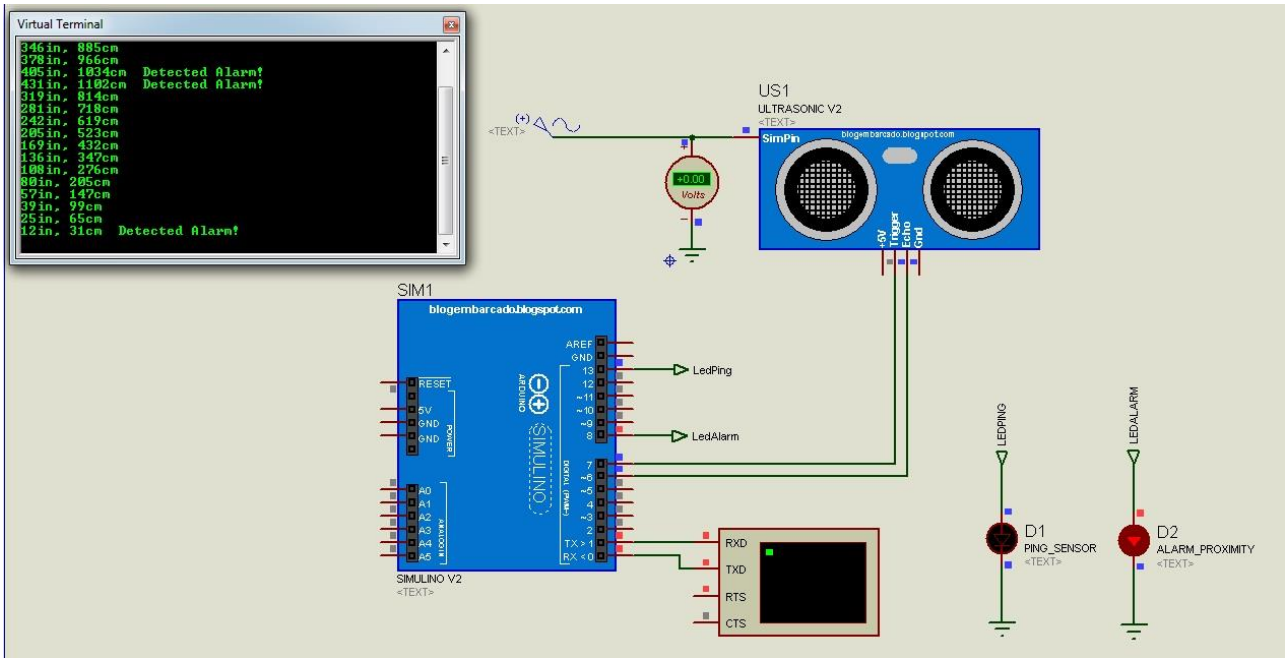
Neste exemplo foi adicionada vários tipos de fontes de tensão disponíveis pelo Proteus. Tanto uma fonte de bateria, quanto uma modelo de fonte dc, e uma fonte de sinal dc através de um modelo virtual.
Cada uma tem um valor diferente, simulando distância diferentes. Então a cada botão pressionado é atualizado o valor da distância lido pelo sensor.

3) UltraSonic Switch



Neste exemplo também foi utilizado vários tipos de modelos de fontes virtuais disponíveis no Proteus. Temos desde fontes DC, assim como uma fonte senoidal com a configuração de um pequeno offset para simular uma fonte DC que varia conforme o tempo, em vista do sensor é uma distância que varia com o tempo, como se houve um objeto se aproximando e afastando continuamente. E também uma fonte de pulsos, que simula um objeto que surge do nada na frente do sensor e no minuto seguinte esse objeto é retirado da frente do mesmo. Se pensarmos em um exemplo prático, poderíamos estar falando de uma esteira que verifica a passagem de uma caixa por exemplo.

4) UltraSonic Proximity



Esse é um exemplo de um alarme com o sensor de proximidade. Ele foi feito para que se o objeto chegar muito próximo de uma distância mínima, ou se ele se afastar de uma distância máxima, ele exibe uma mensagem de alarme, além de acionar um led de aviso. As distâncias podem ser configuradas. E para simular o objeto que se afasta e se aproxima, foi utilizado o modelo da fonte senoidal com o offset necessário, afinal não temos distâncias negativas neste universo. rs

5) Observações.

O novo sensor continua tendo o mesmo funcionamento anterior. Para que o mesmo funcione, basta clicar sobre o mesmo e indicar a localização do arquivo .hex do sensor, chamado "UltraSonicSensor.Hex". A partir dai você pode utilizar qualquer outro microcontrolador para efetuar a leitura da distância simulada.

